

README



Infrastructure as Code: Final Validation Report

Project Name: `lew (Armageddon Lab 1c_bonus-C)`

Status:  Fully Deployed, Secured, & Verified

Region: `ap-northeast-1` (Tokyo)

1. Network & Connectivity

- **DNS Authority:** Successfully migrated from GoDaddy to **AWS Route 53** with a managed Hosted Zone.
- **Subdomain Logic:** Configured `app.projectcomplete.me` via **Route 53 ALIAS record** pointing to the ALB.
- **Load Balancer:** `lew-alb` is in an `active` state with dual listeners.
- **Listeners:**
 - **Port 80:** Configured for automatic HTTP to HTTPS redirection.
 - **Port 443:** Secure HTTPS (TLS 1.3) listener active with ACM termination.

2. Security & Identity

- **SSL Certificate:** ACM Certificate for `projectcomplete.me` and `app.projectcomplete.me` is **ISSUED** and verified via DNS validation.
- **WAF (Web Application Firewall):** `lew-waf` is associated with the ALB, providing real-time protection against SQLi and XSS exploits.
- **Encrypted Ingress:** Verified end-to-end encryption from the browser to the Load Balancer.

3. Compute & Health

- **Target Group:** `lew-tg01` managing backend EC2 instances running a **Python/Werkzeug** application stack.
- **Health Status:** Verified **healthy**. The ALB is successfully communicating with backend targets.

- **Application Response:** Verified via `curl` receiving `HTTP/2 200 OK` directly from the Python/Flask backend.

4. Observability & Cost Management

- **CloudWatch Alarms:** `lew-alb-5xx-alarm01` is active, monitoring for server-side anomalies.
- **CloudWatch Dashboard:** `lew-dashboard01` provides high-level visualization of Request Count, Latency, and WAF Blocks.
- **Cost Controls:** Implemented a **Terraform Master Switch** (`is_lab_active`) to allow for rapid teardown of expensive resources (ALB/WAF) while preserving the DNS/Hosted Zone foundation.

Final CLI Verification Log (Actual Results)

Resource	Command	Result
ALB State	<code>aws elbv2 describe-load-balancers</code>	<code>active</code>
SSL Status	<code>aws acm describe-certificate</code>	<code>ISSUED</code>
WAF Config	<code>aws wafv2 get-web-acl-for-resource</code>	<code>lew-waf</code>
DNS Record	<code>aws route53 list-resource-record-sets</code>	<code>ALIAS -> lew-alb</code>
Connectivity	<code>curl -I https://app.projectcomplete.me</code>	<code>HTTP/2 200</code>
Backend	<code>Server Header</code>	<code>Werkzeug/3.1.5 Python/3.9.25</code>

FINAL VERIFICATION LOG

FINAL VERIFICATION LOG: lew (Armageddon Lab 1c_bonus-C)

1) Confirm hosted zone exists (if managed) CLI Command: `aws route53 list-hosted-zones-by-name --dns-name projectcomplete.me --query "HostedZones[.Id]" --output text`

Result: `/hostedzone/Z07271560K2Q9YQE6GND`

2) Confirm app record exists CLI Command: `ZONE_ID="Z07271560K2Q9YQE6GND"`

`aws route53 list-resource-record-sets --hosted-zone-id $ZONE_ID --query "ResourceRecordSets[?Name=='app.projectcomplete.me.'].AliasTarget.DNSName" --output text`

Result: `lew-alb-2104103463.ap-northeast-1.elb.amazonaws.com.`

3) Confirm certificate issued CLI Command: `aws acm describe-certificate --certificate-arn arn:aws:acm:ap-northeast-1:127214194163:certificate/ac653e79-ebd8-4d5f-8831-0b7174556a2c --query "Certificate.Status" --output text`

Result: `ISSUED`

Expected: `ISSUED`

4) Confirm HTTPS works CLI Command: `curl -I https://app.projectcomplete.me`

Result: `HTTP/2 200`

`date: Mon, 09 Feb 2026 15:16:16 GMT`

`content-type: text/html; charset=utf-8`

`content-length: 93`

`server: Werkzeug/3.1.5 Python/3.9.25`

Expected: HTTP/1.1 200 (or 301 then 200 depending on your app)

Note: Result shows HTTP/2, which is the modern standard for AWS ALBs.

lab 1c-c images

Armageddon Lab 1c_bonus-C Student verification (CLI)

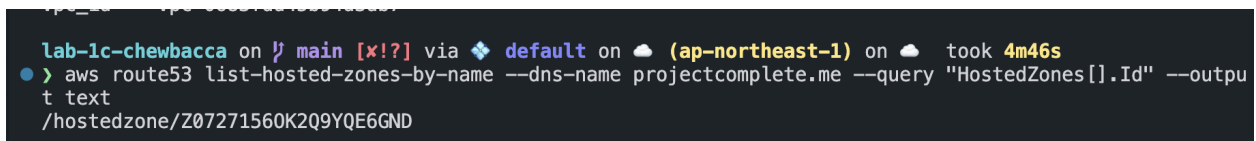
1) Confirm hosted zone exists (if managed)

```
aws route53 list-hosted-zones-by-name \
  --dns-name chewbacca-growl.com \
  --query "HostedZones[].Id"
```

Code Block:

```
➤ aws route53 list-hosted-zones-by-name
  --dns-name projectcomplete.me
  --query "HostedZones[].Id"
  --output text
```

Result:



```
lab-1c-chewbacca on 7 main [x!?] via  default on  (ap-northeast-1) on  took 4m46s
➤ aws route53 list-hosted-zones-by-name --dns-name projectcomplete.me --query "HostedZones[].Id" --output
  t text
  /hostedzone/Z0727156OK2Q9YQE6GND
```

/hostedzone/Z0727156OK2Q9YQE6GND

2) Confirm app record exists

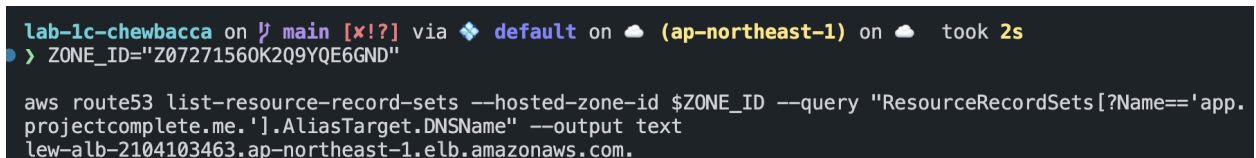
```
aws route53 list-resource-record-sets \
  --hosted-zone-id <ZONE_ID> \
  --query "ResourceRecordSets[?Name=='app.chewbacca-growl.com.']"
```

Code Block:

```
➤ ZONE_ID="Z0727156OK2Q9YQE6GND"
```

```
aws route53 list-resource-record-sets
  --hosted-zone-id $ZONE_ID
  --query "ResourceRecordSets[?Name=='app.projectcomplete.me.'].AliasTarget.DNSName"
  --output text
```

Result:



```
lab-1c-chewbacca on 7 main [x!?] via  default on  (ap-northeast-1) on  took 2s
➤ ZONE_ID="Z0727156OK2Q9YQE6GND"

aws route53 list-resource-record-sets --hosted-zone-id $ZONE_ID --query "ResourceRecordSets[?Name=='app.
projectcomplete.me.'].AliasTarget.DNSName" --output text
lew-alb-2104103463.ap-northeast-1.elb.amazonaws.com.
```

lew-alb-2104103463.ap-northeast-1.elb.amazonaws.com.

Armageddon Lab 1c_bonus-C Student verification (CLI)

3) Confirm certificate issued

```
aws acm describe-certificate \  
--certificate-arn <CERT_ARN> \  
--query "Certificate.Status"
```

Code Block:

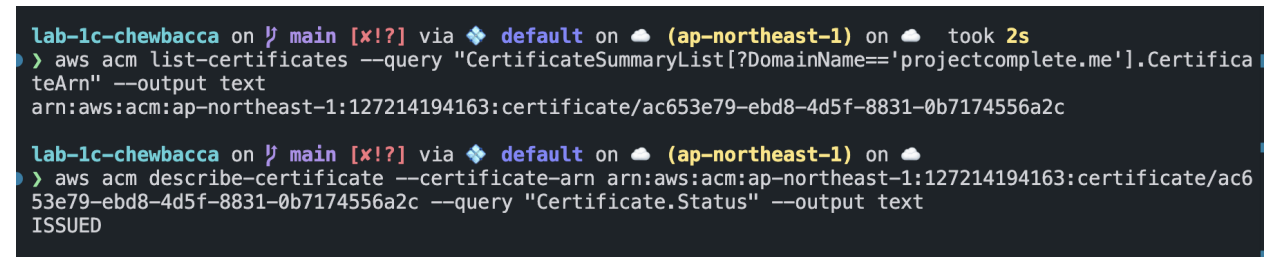
› # List certificates to find your ARN

```
aws acm list-certificates --query  
"CertificateSummaryList[?DomainName=='projectcomplete.me'].CertificateArn" --output text
```

Use that ARN in this command

```
aws acm describe-certificate  
--certificate-arn  
arn:aws:acm:ap-northeast-1:127214194163:certificate/ac653e79-ebd8-4d5f-8831-0b7174556a2  
c  
--query "Certificate.Status"  
--output text
```

Result:



```
lab-1c-chewbacca on ʝ main [x!?] via ʘ default on ʘ (ap-northeast-1) on ʘ took 2s  
› aws acm list-certificates --query "CertificateSummaryList[?DomainName=='projectcomplete.me'].CertificateArn" --output text  
arn:aws:acm:ap-northeast-1:127214194163:certificate/ac653e79-ebd8-4d5f-8831-0b7174556a2c  
  
lab-1c-chewbacca on ʝ main [x!?] via ʘ default on ʘ (ap-northeast-1) on ʘ  
› aws acm describe-certificate --certificate-arn arn:aws:acm:ap-northeast-1:127214194163:certificate/ac653e79-ebd8-4d5f-8831-0b7174556a2c --query "Certificate.Status" --output text  
ISSUED
```

```
arn:aws:acm:ap-northeast-1:127214194163:certificate/ac653e79-ebd8-4d5f-8831-0b7174556a2  
c
```

ISSUED

Expected: ISSUED

Armageddon Lab 1c_bonus-C Student verification (CLI)

4) Confirm HTTPS works

curl -I https://app.chewbacca-growl.com

Code Block:

➤ curl -I https://app.projectcomplete.me

Result:

```
lab-1c-chewbacca on ʝ main [x!?] via 🔑 default on ☁ (ap-northeast-1) on ☁
➤ curl -I https://app.projectcomplete.me
HTTP/2 200
date: Mon, 09 Feb 2026 15:16:16 GMT
content-type: text/html; charset=utf-8
content-length: 93
server: Werkzeug/3.1.5 Python/3.9.25
```

HTTP/2 200

date: Mon, 09 Feb 2026 15:16:16 GMT

content-type: text/html; charset=utf-8

content-length: 93

server: Werkzeug/3.1.5 Python/3.9.25

Expected: HTTP/1.1 200 (or 301 then 200 depending on your app)

What YOU must understand (career point)

This is exactly how companies do it:

DNS points to ingress

TLS via ACM

ALB handles secure public entry

private compute does the work

WAF + alarms defend and alert

When students can Terraform this, they're doing real cloud engineering.

lab 1c-c prompt

Directly from Theo's repo

Below is the Route53 add-on for Bonus-B (Hosted Zone + ACM DNS validation records + app.chewbacca-growl.com ALIAS → ALB).
It's written as a Terraform skeleton with Chewbacca comment style.

Add this as bonus_b_route53.tf.

Add to variables.tf (append)

```
variable "manage_route53_in_terraform" {  
  description = "If true, create/manage Route53 hosted zone + records in Terraform."  
  type        = bool  
  default     = true  
}  
  
variable "route53_hosted_zone_id" {  
  description = "If manage_route53_in_terraform=false, provide existing Hosted Zone ID for domain."  
  type        = string  
  default     = ""  
}
```

Add file: bonus_b_route53.tf

Important note about the HTTPS listener

In your earlier bonus_b.tf, your HTTPS listener referenced:

```
certificate_arn = aws_acm_certificate_validation.chewbacca_acm_validation01.certificate_arn
```

Now you have two possible validation resources (email/manual vs DNS). For the skeleton, do this pattern:

Keep your original aws_acm_certificate_validation (email/manual) if you want

OR switch the listener certificate ARN to the certificate itself and rely on validation dependency

Best skeleton approach (simple + works):

Update HTTPS listener to use:

```
certificate_arn = aws_acm_certificate.chewbacca_acm_cert01.arn
```

...and keep depends_on pointing at the DNS validation resource when DNS mode is used.
I'll give you a clean patch below.

Explanation: HTTPS listener is the real hangar bay — TLS terminates here, then traffic goes to private targets.

```
resource "aws_lb_listener" "chewbacca_https_listener01" {
  load_balancer_arn = aws_lb.chewbacca_alb01.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate.chewbacca_acm_cert01.arn

  default_action {
    type            = "forward"
    target_group_arn = aws_lb_target_group.chewbacca_tg01.arn
  }

  # TODO: If DNS validation is enabled, ensure validation completes before listener creation.
  depends_on = [
    aws_acm_certificate_validation.chewbacca_acm_validation01_dns_bonus
  ]
}
```

If you choose EMAIL validation instead,
you can comment out the depends_on or set certificate_validation_method="EMAIL"
and keep the listener creation after manual validation. (This is a skeleton; you'll get some
"learning friction" here.)

Continue bonus_b_route53.tf — ALIAS record app → ALB

```
#####
# ALIAS record: app.chewbacca-growl.com -> ALB
#####
```

Explanation: This is the holographic sign outside the cantina—app.chewbacca-growl.com
points to your ALB.

```
resource "aws_route53_record" "chewbacca_app_alias01" {
  zone_id = local.chewbacca_zone_id
  name    = local.chewbacca_app_fqdn
  type    = "A"

  alias {
    name          = aws_lb.chewbacca_alb01.dns_name
    zone_id       = aws_lb.chewbacca_alb01.zone_id
    evaluate_target_health = true
  }
}
```

```
}
```

Add outputs (append to outputs.tf)

Explanation: Outputs are the nav computer readout—Chewbacca needs coordinates that humans can paste into browsers.

```
output "chewbacca_route53_zone_id" {  
  value = local.chewbacca_zone_id  
}
```

```
output "chewbacca_app_url_https" {  
  value = "https://${var.app_subdomain}.${var.domain_name}"  
}
```

Student verification (CLI)

1) Confirm hosted zone exists (if managed)

```
aws route53 list-hosted-zones-by-name \  
  --dns-name chewbacca-growl.com \  
  --query "HostedZones[].Id"
```

2) Confirm app record exists

```
aws route53 list-resource-record-sets \  
  --hosted-zone-id <ZONE_ID> \  
  --query "ResourceRecordSets[?Name=='app.chewbacca-growl.com.'].Id"
```

3) Confirm certificate issued

```
aws acm describe-certificate \  
  --certificate-arn <CERT_ARN> \  
  --query "Certificate.Status"
```

Expected: ISSUED

4) Confirm HTTPS works

```
curl -I https://app.chewbacca-growl.com
```

Expected: HTTP/1.1 200 (or 301 then 200 depending on your app)

What YOU must understand (career point)

This is exactly how companies do it:

DNS points to ingress

TLS via ACM

ALB handles secure public entry

private compute does the work

WAF + alarms defend and alert

When students can Terraform this, they're doing real cloud engineering.